

به نام خدا

راهنمای استفاده از کتابخانه های بینایی کامپیوتر در پایتون

گردآورندگان:

سید محمد حسین مظهري

سید محمد جزایري

سید نوید هاشمی

فهرست مطالب

فصل اول) کتابخانه Pillow

page 3

فصل دوم) کتابخانه OpenCV

page 12

فصل سوم) سایر کتابخانه ها

page 23

فصل چهارم) الگوریتم های بینایی

page 37

فصل پنجم) منابع

page 42

فصل اول

Pillow کتابخانه

معرفی Pillow

کتابخانه Pillow نسخه به‌روزرسانی‌شده‌ی کتابخانه‌ی PIL است که برای پردازش تصاویر در زبان پایتون استفاده می‌شود. این کتابخانه ابزارهایی برای باز کردن، ویرایش، فیلترگذاری، رسم، تبدیل رنگ و ذخیره تصاویر ارائه می‌دهد.

نصب و راه‌اندازی

برای نصب کتابخانه در محیط‌های مختلف، از دستورات زیر استفاده می‌کنیم:

محیط پایتون: `pip install pillow`

محیط نوت‌بوک: `pip install pillow!`

پس از اینکه کتابخانه با موفقیت نصب شد، برای استفاده، باید آن را با دستور روبه‌رو import کنیم:

```
from PIL import Image, ImageFilter, ImageDraw, ImageFont
```

لیست توابع و کلاس‌های مهم

ماژول پایه‌ای: `Image`

توضیح مختصر	تابع / کلاس
باز کردن تصویر از مسیر مشخص	<code>Image.open()</code>
ساخت یک تصویر جدید با رنگ و اندازه دلخواه	<code>Image.new()</code>
ذخیره تصویر در فایل	<code>Image.save()</code>
نمایش تصویر با نرم‌افزار پیش‌فرض سیستم	<code>Image.show()</code>
ایجاد یک کپی از تصویر	<code>Image.copy()</code>

<code>Image.close()</code>	بستن تصویر برای آزاد کردن منابع سیستم
<code>Image.format</code>	نمایش فرمت فایل تصویری (مثل PNG, JPEG)
<code>Image.mode</code>	نمایش حالت رنگ تصویر (مثل RGB, L, CMYK)
<code>Image.size</code>	اندازه تصویر را برمیگرداند (عرض، ارتفاع به پیکسل)
تابع	توضیح مختصر
<code>Image.resize()</code>	تغییر اندازه تصویر به ابعاد مشخص
<code>Image.rotate()</code>	چرخش تصویر به زاویه داده شده
<code>Image.transpose()</code>	معکوس کردن تصویر افقی/عمودی یا چرخش ۹۰/۱۸۰/۲۷۰
<code>Image.crop()</code>	برش بخشی از تصویر
<code>Image.thumbnail()</code>	تولید تصویر بندانگشتی با حفظ نسبت تصویر

فیلترگذاری و افکت ها (از `ImageFilter`)

فیلتر	توضیح مختصر
<code>BLUR</code>	محو کردن تصویر (افکت بلور)
<code>CONTOUR</code>	نمایش خطوط و لبه ها
<code>DETAIL</code>	افزایش جزئیات تصویر
<code>EDGE_ENHANCE</code>	برجسته سازی لبه ها
<code>EMBOSS</code>	ایجاد افکت برجسته
<code>SHARPEN</code>	تیز کردن تصویر
<code>SMOOTH</code>	صاف کردن تصویر

دستکاری رنگ و کانال

توضیح مختصر	تابع / متد
تبدیل مدل رنگی (مثلاً از RGB به L برای سیاه سفید)	<code>()convert</code>
جدا کردن کانال‌های رنگ (مثلاً R, G, B)	<code>()split</code>
ترکیب کانال‌ها به یک تصویر رنگی	<code>()merge</code>
گرفتن رنگ یک پیکسل خاص	<code>()getpixel</code>
تغییر رنگ یک پیکسل خاص	<code>()putpixel</code>

ترسیم و متن (از `ImageDraw` , `ImageFont`)

توضیح مختصر	ابزار
آماده‌سازی تصویر برای ترسیم	<code>()ImageDraw.Draw</code>
رسم خط روی تصویر	<code>()draw.line</code>
رسم مستطیل روی تصویر	<code>()draw.rectangle</code>
نوشتن متن روی تصویر	<code>()draw.ellipse</code>
تغییر رنگ یک پیکسل خاص	<code>()draw.text</code>
انتخاب فونت سفارشی برای نوشتن متن	<code>()ImageFont.truetype</code>

کار با آلفا، شفافیت، ماسک

توضیح مختصر	تابع
اضافه کردن کانال شفافیت (alpha) به تصویر	<code>()putalpha</code>
ترکیب دو تصویر با آلفا	<code>Image.alpha_composite</code>
اعمال تغییر به پیکسل‌ها با تابع دلخواه	<code>()Image.point</code>

حالت‌های تصویر (Modes)

mode	توضیح مختصر
1	سیاه و سفید (1 بیت)
L	خاکستری (8 بیت)
RGB	رنگی (24 بیت)
RGBA	رنگی با شفافیت
CMYK	چاپ حرفه‌ای (رنگ چهارگانه)
P	پالت رنگ

نمونه کدها و خروجی‌های کاربردی

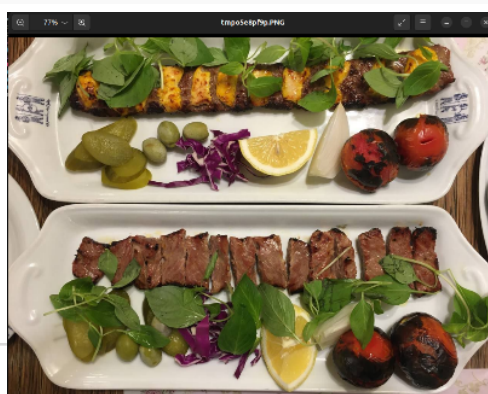
۱. باز کردن، نمایش و ذخیره تصویر

```
from PIL import Image

# باز کردن تصویر از مسیر مشخص
img = Image.open("image.jpg") # مسیر تصویر را به نام تصویر خودتان تغییر دهید ←

# نمایش تصویر
img.show()

# ذخیره تصویر با نام جدید
img.save("output open save.jpg")
```



۲. تغییر اندازه تصویر (resize)

```
from PIL import Image

img = Image.open("image.jpg")

# تغییر اندازه تصویر به ابعاد جدید (عرض 400، ارتفاع 300)
resized_img = img.resize((400, 300))

# نمایش و ذخیره تصویر جدید
resized_img.show()
resized_img.save("output_resize.jpg")
```

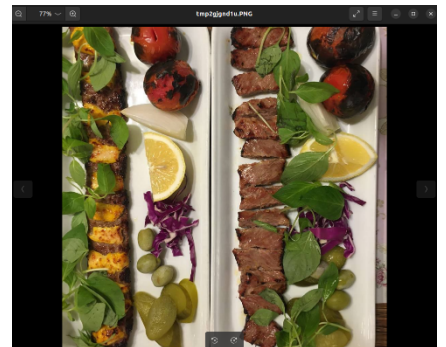
۳. چرخش تصویر (rotate)

```
from PIL import Image

img = Image.open("image.jpg")

# چرخش تصویر به اندازه 90 درجه خلاف جهت عقربه‌های ساعت
rotated_img = img.rotate(90)

# نمایش و ذخیره تصویر چرخیده شده
rotated_img.show()
rotated_img.save("output_rotate.jpg")
```



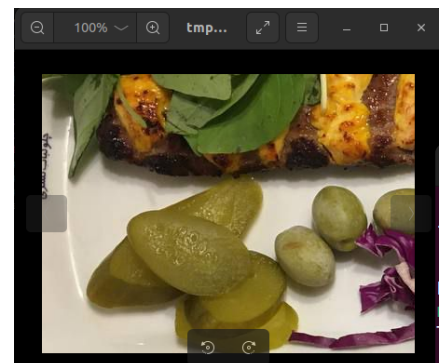
۴. برش تصویر (crop)

```
from PIL import Image

img = Image.open("image.jpg")

# برش قسمت مشخصی از تصویر با مختصات (چپ، بالا، راست، پایین)
cropped_img = img.crop((100, 100, 500, 400))

# نمایش و ذخیره خروجی
cropped_img.show()
cropped_img.save("output_crop.jpg")
```



۵. اعمال فیلترها (ImageFilter)

```

from PIL import Image, ImageFilter

img = Image.open("image.jpg")

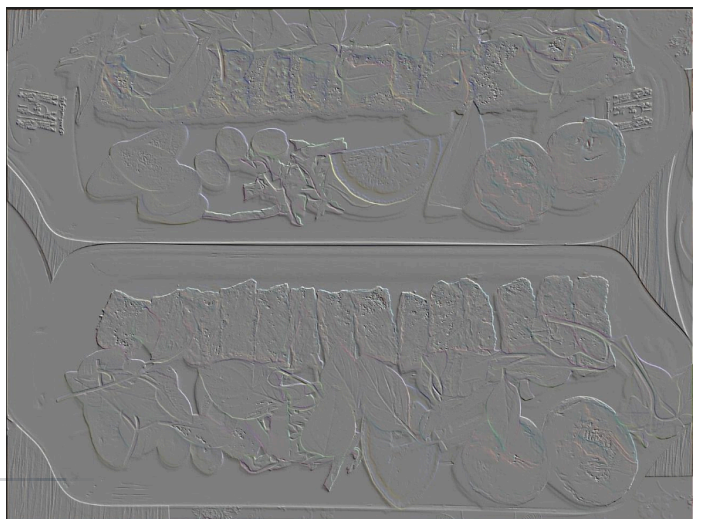
# اعمال فیلتر محو (Blur)
blurred = img.filter(ImageFilter.BLUR)
blurred.save("output_blur.jpg")

# اعمال فیلتر برجسته‌سازی (Sharpen)
sharpened = img.filter(ImageFilter.SHARPEN)
sharpened.save("output_sharpen.jpg")

# اعمال فیلتر امپاس (Emboss)
embossed = img.filter(ImageFilter.EMBOSS)
embossed.save("output_emboss.jpg")

```

خروجی های قرار داده شده به ترتیب فیلترهای
اعمال شده قرار گرفته اند.
(Blur, Sharpen, Emboss) از چپ به راست



۶. تبدیل رنگ تصویر (Grayscale و RGBA)-----

```
from PIL import Image

img = Image.open("image.jpg")

# تبدیل به سیاه و سفید (Grayscale)
gray_img = img.convert("L")
gray_img.save("output_grayscale.jpg")

# تبدیل به تصویر دارای شفافیت (RGBA)
rgba_img = img.convert("RGBA")
rgba_img.save("output_rgba.jpg")
```



۷. نوشتن متن روی تصویر (text)-----

```
from PIL import Image, ImageDraw, ImageFont

img = Image.open("image.jpg")

# آماده‌سازی برای رسم
draw = ImageDraw.Draw(img)

# بارگذاری فونت (در صورت نداشتن، این خط را حذف کنید)
# font = ImageFont.truetype("arial.ttf", 36)

# نوشتن متن ساده روی تصویر
draw.text((50, 50), "سلام دنیا", fill="red") # رنگ، متن، موقعیت

# نمایش و ذخیره
img.show()
img.save("output_text.jpg")
```



۸. رسم اشکال (دایره، مستطیل، خط)-----

```
from PIL import Image, ImageDraw

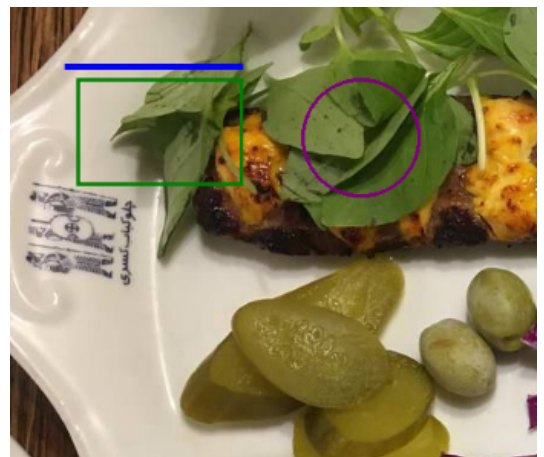
img = Image.open("image.jpg")
draw = ImageDraw.Draw(img)

# رسم خط از نقطه (x1, y1) تا (x2, y2)
draw.line((50, 50, 200, 50), fill="blue", width=5)

# رسم مستطیل
draw.rectangle((60, 60, 200, 150), outline="green", width=3)

# رسم دایره (بیضی در کادر مربعی)
draw.ellipse((250, 60, 350, 160), outline="purple", width=3)

# نمایش و ذخیره
img.show()
img.save("output_shapes.jpg")
```



۹. جدا کردن کانال‌های رنگ (RGB)

```
from PIL import Image

img = Image.open("image.jpg")

# جدا کردن کانال‌های قرمز، سبز، آبی
r, g, b = img.split()

# ذخیره هر کانال به صورت تصویر جداگانه
r.save("output_r.jpg")
g.save("output_g.jpg")
b.save("output_b.jpg")
```

فصل دوم

OpenCV کتابخانه

معرفی کتابخانه openCV

کتابخانه OpenCV (مخفف Open Source Computer Vision Library) یک کتابخانه متن باز و چند پلتفرمی برای بینایی ماشین و پردازش تصویر است که در ابتدا توسط شرکت Intel توسعه داده شد و امروزه توسط جامعه‌ای بزرگ از برنامه‌نویسان و پژوهشگران پشتیبانی می‌شود.

OpenCV در زبان‌های مختلفی مانند Java، Python، C++ و حتی JavaScript قابل استفاده است، اما نسخه پایتون آن به دلیل سادگی و سرعت توسعه، بسیار پرکاربردتر است.

نصب و راه اندازی

برای نصب این کتابخانه در پایتون ورژن 3 از دستور `pip3 install opencv-python` استفاده می‌کنیم

نکته 1: اگر پایتون شما ورژن 2 است به جای `pip3` فقط `pip` بنویسید

نکته 2: اگر در محیطی مانند `jupyter notebook` می‌خواهید نصب کنید در ابتدای دستور ! قرار دهید

برای استفاده از کتابخانه در محیط پایتون فقط کافیست این تکه کد را به کد خود اضافه کنید

```
import cv2
```

لیست ماژول های مهم

نام ماژول	کاربرد
<code>cv2.imgcodecs</code>	خواندن و نوشتن فایل‌های تصویری
<code>cv2.imgproc</code>	پردازش تصویر (فیلترها، تبدیل رنگ، آشکارسازی لبه و ...)
<code>cv2.highgui</code>	نمایش تصویر و ایجاد پنجره‌های گرافیکی

ترمیم تصویر، inpainting، حذف نویز	cv2.photo
شناسایی و توصیف ویژگی‌ها مانند SIFT، ORB و ...	cv2.features2d
تشخیص اشیاء مانند چهره، چشم، بدن و ...	cv2.objdetect

توابع پایه imgcodecs

توضیح مختصر	تابع/کلاس
خواندن تصویر از مسیر مشخص	cv2.imread()
ذخیره تصویر در مسیر داده شده	cv2.imwrite()
پارامترهای انتخابی برای خواندن تصویر به صورت رنگی یا سیاه‌سفید	cv2.IMREAD_COLOR / GRAYSCALE

توابع پایه imgproc

توضیح مختصر	تابع/کلاس
تبدیل مدل رنگ (مثلاً BGR به Gray)	cv2.cvtColor()
اعمال فیلتر بلور گاوسی	cv2.GaussianBlur()
تشخیص لبه‌ها با الگوریتم Canny	cv2.Canny()

<code>cv2.threshold()</code>	آستانه‌گذاری برای باینری کردن تصویر
<code>cv2.adaptiveThreshold()</code>	آستانه‌گذاری تطبیقی
<code>cv2.erode()</code> / <code>cv2.dilate()</code>	فرسایش یا گسترش اجزای تصویر
<code>cv2.resize()</code>	تغییر اندازه تصویر
<code>cv2.flip()</code>	چرخاندن تصویر (افقی، عمودی)
<code>cv2.getRotationMatrix2D()</code> + <code>warpAffine()</code>	چرخاندن تصویر به زاویه دلخواه

توابع پایه highgui

توضیح مختصر	تابع/کلاس
نمایش تصویر در یک پنجره جدید (ویندوزی)	<code>cv2.imshow()</code>
انتظار برای زدن کلید توسط کاربر	<code>cv2.waitKey()</code>
بستن تمام پنجره‌های باز شده	<code>cv2.destroyAllWindows()</code>
ساخت پنجره با نام مشخص	<code>cv2.namedWindow()</code>
تنظیم تابع بازخورد برای ماوس	<code>cv2.setMouseCallback()</code>

توابع پایه photo

توضیح مختصر	تابع / متد
-------------	------------

<code>cv2.fastNlMeansDenoising()</code>	حذف نویز از تصویر خاکستری
<code>cv2.fastNlMeansDenoisingColored()</code>	حذف نویز از تصویر رنگی
<code>cv2.inpaint()</code>	ترمیم ناحیه مشخص شده با ماسک

توابع پایه feature2d

توضیح مختصر	تابع / متد
ایجاد تشخیص دهنده ویژگی ORB	<code>cv2.ORB_create()</code>
ایجاد تشخیص دهنده ویژگی SIFT (نیاز به OpenCV با مجوز non-free)	<code>cv2.SIFT_create()</code>
تشخیص کلیدنقاط و محاسبه توصیف گرها	<code>detector.detectAndCompute()</code>
رسم کلیدنقاط روی تصویر	<code>cv2.drawKeypoints()</code>
مقایسه و تطبیق توصیف گرها (Brute Force)	<code>cv2.BFMatcher()</code>

توابع پایه objdetect

توضیح مختصر	تابع / متد
بارگذاری طبقه بند Haar	<code>cv2.CascadeClassifier()</code>
تشخیص چهره یا اشیاء در تصویر	<code>CascadeClassifier.detectMultiScale()</code>

حالات رنگی مهم

توضیح مختصر	نام حالت
تبدیل تصویر رنگی به سیاه و سفید	<code>cv2.COLOR_BGR2GRAY</code>
تبدیل BGR به RGB (برای سازگاری با matplotlib)	<code>cv2.COLOR_BGR2RGB</code>
تبدیل به فضای رنگ HSV	<code>cv2.COLOR_BGR2HSV</code>
تبدیل به فضای رنگ LAB	<code>cv2.COLOR_BGR2LAB</code>

نکته: در OpenCV، به طور پیش فرض تصاویر در فضای BGR هستند نه RGB. فرق این دو فقط در ترتیب رنگ هاست وگرنه رنگ های اصلی همان قرمز سبز و آبی هستند.

حال با استفاده از این توابع چند نمونه کد جالب به همراه خروجی خواهیم دید.

ماژول `imgcodecs`

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("kid1.jpg")
cv2_imshow(img)

cv2.imwrite("saved_image.png", img)
```

با استفاده از این کد تصویر را می خوانیم سپس نمایش می دهیم سپس ذخیره می کنیم.
خروجی در پایین قابل مشاهده است.



ماژول `imgproc`

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("kid1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)
cv2_imshow(edges)
```

در این بخش ابتدا تصویر را خاکستری می‌کنیم و سپس با فیلتر `canny` مرزها را تشخیص می‌دهیم.



ماژول photo

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("kid1.jpg")
denoised = cv2.fastNlMeansDenoisingColored(img, None, 10, 10, 7, 21)
cv2_imshow(denoised)
```

این تابع نویز را برای تصاویر رنگی حذف می‌کند و همانگونه که در تصویر پایین قابل مشاهده است کیفیت تصویر بهبود یافته است.



ماژول features2d

```
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("kid1.jpg")
orb = cv2.ORB_create()
kp, des = orb.detectAndCompute(img, None)
out = cv2.drawKeypoints(img, kp, None, color=(0, 255, 0))
cv2_imshow(out)
```

این قطعه کد با استفاده از الگوریتم بینایی ORB که در فصل 4 تشریح خواهد شد مهمترین مشخصه ها (features) یک تصویر را می یابد و با نقطه مشخص می کند.



ماژول objdetect

```
import cv2
from google.colab.patches import cv2_imshow

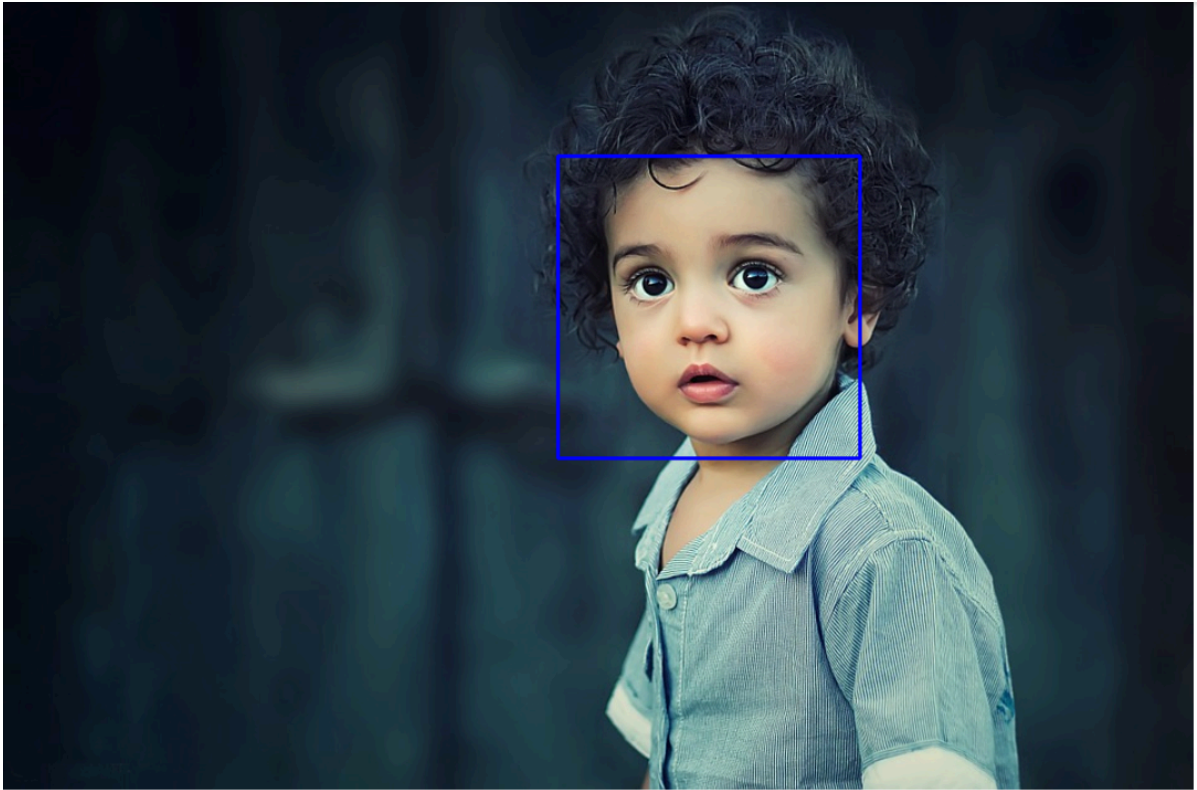
img = cv2.imread("kid1.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + "haarcascade_frontalface_default.xml")
faces = face_cascade.detectMultiScale(gray, 1.3, 5)

for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x+w, y+h), (255, 0, 0), 2)

cv2_imshow(img)
```

این قطعه کد با استفاده از الگوریتم Haar که در فصل 4 تشریح خواهد شد وجود یک شیء (object) را تشخیص می‌دهد و دور آن یک مستطیل می‌کشد.



فصل سوم

سایر کتابخانه ها

کتابخانه scikit-image

کتابخانه‌ی scikit-image مجموعه‌ای از الگوریتم‌های متن‌باز برای پردازش تصویر در پایتون است scikit-image.org. این پکیج روی آرایه‌های NumPy کار می‌کند و امکاناتی مانند خواندن تصویر، تبدیل رنگ، فیلترگذاری، برش (crop)، تغییر اندازه و تشخیص ویژگی‌ها (مانند لبه‌ها و گوشه‌ها) را فراهم می‌کند. scikit-image به عنوان یک تکمیل‌کننده‌ی OpenCV شناخته شده و برای کارهای مختلف بینایی کامپیوتر مثل پیش‌پردازش تصویر، تشخیص لبه، تقسیم‌بندی و آنالیز داده‌های تصویری کاربرد دارد.

نصب و راه‌اندازی

در محیط پایتون می‌توانید با دستور زیر scikit-image را نصب کنید:

```
pip install scikit-image
```

در محیط نوت بوک:

```
!pip install scikit-image
```

توابع و ماژول‌های مهم

در scikit-image بسیاری از توابع در زیرماژول‌های مختلف قرار دارند. از مهم‌ترین آن‌ها می‌توان به موارد زیر اشاره کرد:

تابع / متد	توضیح مختصر
<code>skimage.io.imread(path)</code>	خواندن تصویر از یک فایل و بازگشت به صورت آرایه‌ی NumPy
<code>skimage.io.imshow(image)</code>	نمایش تصویر (اغلب برای پیش‌نمایش)
<code>skimage.io.imwrite(path, image)</code>	ذخیره‌ی تصویر (آرایه‌ی NumPy) در فایل.
<code>skimage.color.rgb2gray(image)</code>	تبدیل تصویر رنگی به تصویر خاکستری
<code>skimage.filters.sobel(image)</code>	اعمال فیلتر سوبل برای تشخیص لبه‌ها
<code>skimage.filters.gaussian(image, sigma)</code>	اعمال فیلتر گوسی (برای بلور کردن تصویر)

تغییر اندازه‌ی تصویر به ابعاد مشخص.	<code>skimage.transform.resize(image, output_shape)</code>
چرخش تصویر به زاویه دلخواه (درجه)	<code>skimage.transform.rotate(image, angle)</code>
عملیات ریخت‌شناسی مانند گسترش (dilation) یا خوردگی (erosion)	<code>skimage.morphology.dilation(image)</code>
برچسب‌گذاری (labeling) بخش‌های متصل (برای تقسیم‌بندی اجزاء)	<code>skimage.measure.label(image)</code>

نمونه کد

```
import skimage.io as io
import skimage.color as color
import skimage.filters as filters

# خواندن تصویر رنگی
image = io.imread("path/to/image.jpg")

# تبدیل تصویر به مقیاس خاکستری
gray = color.rgb2gray(image)

# اعمال فیلتر سوبل برای تشخیص لبه‌ها
edges = filters.sobel(gray)

# نمایش تصویر حاصل (لبه‌ها)
io.imshow(edges)
io.show()
```

کتابخانه imageio

کتابخانه‌ی **imageio** یک رابط ساده برای خواندن و نوشتن انواع فرمت‌های تصویری و ویدئویی در پایتون ارائه می‌دهد imageio.readthedocs.io. با استفاده از imageio

می‌توانید تصاویر ایستا (مثل JPEG, PNG)، تصاویر متحرک (مثل GIF)، ویدئوها و داده‌های حجمی علمی را به راحتی بارگذاری و ذخیره کنید. این کتابخانه مستقل از سیستم‌عامل است و از بسیاری از فرمت‌های رایج تصویر پشتیبانی می‌کند.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install imageio
```

در محیط نوت بوک:

```
!pip install imageio
```

توابع و ماژول‌های مهم

توضیح مختصر	تابع / متد
خواندن تصویر از فایل یا آدرس و بازگشت به صورت آرایه‌ی NumPy	<code>imageio.imread(uri)</code>
ذخیره‌ی آرایه‌ی NumPy به عنوان تصویر در فایل	<code>imageio.imwrite(uri, image)</code>
ذخیره‌ی یک رشته از تصاویر به عنوان GIF یا ویدئو (از فرمت‌های پشتیبانی‌شده)	<code>imageio.mimsave(uri, sequence)</code>
ایجاد شیئی برای خواندن فریم‌های یک ویدئو یا فایل انیمیشن	<code>imageio.get_reader(uri)</code>
ایجاد شیئی برای نوشتن فریم‌ها در یک ویدئو یا فایل انیمیشن	<code>imageio.get_writer(uri)</code>

```
import imageio

# خواندن تصویر از فایل
img = imageio.imread("path/to/photo.png")

# ذخیره تصویر در فایل جدید
imageio.imwrite("path/to/photo_copy.png", img)

# مثال: خواندن فریم‌های یک ویدئو
reader = imageio.get_reader("path/to/video.mp4")
for frame in reader:
    # در اینجا می‌توانید روی هر فریم پردازش انجام دهید
    print(frame.shape)
    break
```

نمونه کد

کتابخانه Matplotlib

Matplotlib یک کتابخانه‌ی جامع برای ایجاد نمودارها و تصاویر ایستا، متحرک و تعاملی در پایتون است github.com. اگرچه Matplotlib در اصل برای رسم نمودار طراحی شده، اما معمولاً برای نمایش تصاویر نیز از آن استفاده می‌شود؛ به‌ویژه تابع **imshow** برای نمایش آرایه‌های تصویری کاربردی است. Matplotlib تصاویر با کیفیت انتشاراتی (publication quality) تولید می‌کند و می‌تواند در اسکرپت، شل‌های پایتون/IPython، و اپلیکیشن‌های وب یا دسکتاپ به کار رود github.com.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install matplotlib
```

در محیط نوت بوک:

```
!pip install matplotlib
```

توابع و ماژول‌های مهم

توضیح مختصر	تابع / متد
ایجاد یک شکل (figure) جدید برای رسم نمودار یا تصویر	<code>plt.figure()</code>
نمایش تصویر دوبعدی (آرایه‌ی NumPy) با یک نگاشت رنگی (cmap)	<code>plt.imshow(image, cmap)</code>
افزودن عنوان به تصویر یا نمودار	<code>plt.title(text)</code>
خاموش کردن محورهای مختصات (برای نمایش فقط تصویر)	<code>plt.axis('off')</code>
نمایش یک نوار رنگی برای مقیاس رنگ (مناسب تصاویر مقیاس خاکستری یا داده‌ها)	<code>plt.colorbar()</code>
نمایش نهایی شکل ایجاد شده روی صفحه (بخش استاندارد نمایش)	<code>plt.show()</code>
ذخیره‌ی شکل فعلی به صورت فایل تصویری (مثلاً PNG, PDF)	<code>plt.savefig(file)</code>

نمونه کد

```
import matplotlib.pyplot as plt
import numpy as np

# ایجاد یک تصویر نمونه (شیب تند خاکستری)
image = np.tile(np.linspace(0, 1, 256), (256, 1))

plt.figure()
plt.imshow(image, cmap='gray') # نمایش تصویر در نگاشت خاکستری
plt.axis('off') # مخفی کردن محورهای مختصات
plt.title("Gradient Image")
plt.show()
```

کتابخانه Albumentations

کتابخانه‌ی **Albumentations** ابزاری سریع و انعطاف‌پذیر برای افزایش داده‌های تصویری (**Image Augmentation**) در یادگیری عمیق و بینایی کامپیوتر است. albumentations.ai. این کتابخانه انواع تبدیلات رایج مانند چرخش، برش تصادفی، برش ابعاد، تغییر روشنایی/کنتراست و غیره را به سرعت روی تصاویر اعمال می‌کند. Albumentations با فراهم کردن یک چارچوب قدرتمند برای زنجیره‌سازی (Compose) تبدیلات، به‌ویژه در مسائل طبقه‌بندی، تقسیم‌بندی و تشخیص اشیاء تصویری کاربرد دارد albumentations.ai.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install albumentations
```

در محیط نوت بوک:

```
!pip install albumentations
```


توابع و ماژول های مهم

توضیح مختصر	تابع / متد
وارونگی افقی تصویر با احتمال ۰/۵	<code>augmentations.HorizontalFlip(p=0.5)</code>
برش تصادفی در ابعاد مشخص	<code>augmentations.RandomCrop(width, height)</code>
چرخش تصویر در محدوده زاویه های دلخواه	<code>augmentations.Rotate(limit)</code>
تغییر تصادفی روشنایی و کنتراست	<code>augmentations.RandomBrightnessContrast()</code>
نرمال سازی تصویر با میانگین و انحراف معیار مشخص (برای آماده سازی ورودی مدل)	<code>augmentations.Normalize()</code>
تغییر اندازه تصویر	<code>augmentations.Resize(width, height)</code>

ساخت کامپوزیت از این تبدیلات:

```
transform = augmentations.Compose([
    HorizontalFlip(p=0.5),
    RandomBrightnessContrast(p=0.2),
    Resize(224, 224)
])
```

نمونه کد

```
import augmentations as A
import cv2

# تعریف یک زنجیره از تبدیلات افزایشی
transform = A.Compose([
    A.RandomCrop(width=256, height=256),
    A.HorizontalFlip(p=0.5),
    A.RandomBrightnessContrast(p=0.2),
])

# خواندن تصویر با OpenCV (BGR)
image = cv2.imread("path/to/image.jpg")
```

```
# اعمال تبدیلات (خروجی به صورت دیکشنری شامل تصویر جدید)
augmented = transform(image=image)
image_aug = augmented['image']

# نمایش تصویر اصلی و افزوده شده (برای مشاهده باید محیط گرافیکی داشته باشید)
cv2.imshow("Original", image)
cv2.imshow("Augmented", image_aug)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

کتابخانه Torchvision

Torchvision بخشی از اکوسیستم PyTorch است که ابزارها و دیتاست‌های پرکاربرد بینایی کامپیوتر را فراهم می‌کند docs.pytorch.org. این پکیج شامل ماژول‌های زیر است:

ماژول	توضیح مختصر
torchvision.datasets	مجموعه‌ای از دیتاست‌های آماده (مانند CIFAR, ImageNet, MNIST و غیره).
torchvision.transforms	تبدیل‌های رایج روی تصاویر (مانند تغییر اندازه، نرمال‌سازی، تبدیل به تانسور) که می‌توان آن‌ها را در یک مسیر انتقال (Compose) زنجیره‌ای کرد.
torchvision.models	مدل‌های پیش‌آموزش‌یافته (pretrained) برای کارهای مختلف (مثل VGG، ResNet، MobileNet و غیره).
torchvision.io	توابع ورودی/خروجی تصاویر و ویدئوها.
torchvision.utils	ابزارهایی برای پردازش دید کامپیوتر، مانند رسم باکس‌های تشخیص و ساخت گرید از تصاویر.

در یک نگاه کلی، torchvision ابزارهایی را فراهم می‌کند تا به راحتی داده‌های تصویری را بارگذاری کرده و روی آن‌ها پیش‌پردازش انجام دهید، یا از مدل‌های از پیش‌آموزش‌دیده استفاده کنید docs.pytorch.org.

نصب و راه اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید (معمولاً به همراه نصب PyTorch می‌آید):

```
pip install torchvision
```

ماژول ها و کلاس های مهم

توضیح مختصر	ماژول
بارگذاری دیتاست‌های مرسوم.	<code>datasets.ImageNet</code> <code>datasets.CIFAR10</code> <code>datasets.FakeData</code>
تبدیل‌های متداول.	<code>transforms.Resize((H, W))</code> <code>transforms.ToTensor()</code> <code>transforms.Normalize()</code> <code>transforms.RandomHorizontalFlip()</code>
ترکیب کردن چندین تبدیل به یک مسیر (pipeline).	<code>transforms.Compose([...])</code>
مدل‌های از پیش‌آموزش‌دیده.	<code>models.resnet18(pretrained=True)</code>
برای نمایش دسته‌ای از تصاویر.	<code>utils.make_grid()</code> <code>utils.save_image()</code>

نمونه کد

```
import torch
from torchvision import datasets, transforms, models

# تبدیل به تانسور 224x224 تعریف مسیر تبدیل داده: تغییر اندازه به 224
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

```
# با تبدیلات (FakeData) بارگزاری یک دیتاست فرضی
dataset = datasets.FakeData(transform=transform)
image, label = dataset[0] # گرفتن اولین تصویر و برچسب آن

# استفاده از یک مدل از پیش آموزش‌دیده ResNet-18
model = models.resnet18(pretrained=True)
output = model(image.unsqueeze(0)) # افزودن بعد بچ (batch dimension)

print(output.shape) # خروجی مدل برای یک تصویر
```

کتابخانه MediaPipe

MediaPipe یک چارچوب متن‌باز شرکت گوگل برای ایجاد و اجرای زنجیره‌های یادگیری ماشین چندرسانه‌ای (مانند تصویر، ویدئو و صوت) است quickpose.ai. این کتابخانه با فراهم آوردن مجموعه‌ای از راه‌حل‌های آماده (مانند تشخیص چهره، شناسایی نقاط اعضای بدن، شناسایی دست و غیره) امکان اجرای بلادرنگ (real-time) وظایف بینایی کامپیوتر را روی پلتفرم‌های مختلف (دسکتاپ، موبایل، وب و غیره) به‌سادگی فراهم می‌کند. MediaPipe بهینه‌شده برای پردازش هم‌زمان (real-time) بوده و برای کارهای شناسایی ویژگی‌های انسانی (چهره، دست‌ها، اسکلتی) کاربرد فراوانی دارد quickpose.ai.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install mediapipe
```

ماژول‌ها و کلاس‌های مهم

در MediaPipe، اکثر قابلیت‌های کاربردی در زیرماژول‌های mediapipe.solutions قرار دارند. برخی از مهم‌ترین آن‌ها عبارت‌اند از:

ماژول	توضیح مختصر
<code>mediapipe.solutions.face_detection.FaceDetection</code>	تشخیص چهره در تصویر یا فریم ویدئو.
<code>mediapipe.solutions.hands.Hands</code>	شناسایی نقاط کلیدی دست و ردیابی دست‌ها.
<code>mediapipe.solutions.pose.Pose</code>	تشخیص وضعیت بدنی (Pose) انسان (مفاصل اصلی بدن).
<code>mediapipe.solutions.face_mesh.FaceMesh</code>	یافتن شبکه‌ای از نقاط دوبعدی و سه‌بعدی روی چهره.

این کلاس‌ها معمولاً در یک بلاک `with` به کار می‌روند تا با خواندن تصویر و پردازش، خروجی داده‌های شناسایی را بدهند.

نمونه کد

```
import cv2
import mediapipe as mp

mp_face = mp.solutions.face_detection

# برای تشخیص چهره FaceDetection ایجاد می‌شود
with mp_face.FaceDetection(model_selection=1, min_detection_confidence=0.5) as face_detector:
    image = cv2.imread("path/to/image.jpg")
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    # پردازش تصویر برای تشخیص چهره
    results = face_detector.process(image_rgb)

    # اگر چهره‌ای پیدا شده باشد، دور آن مستطیل بکش
    if results.detections:
        ih, iw, _ = image.shape
        for detection in results.detections:
            bboxC = detection.location_data.relative_bounding_box
            x = int(bboxC.xmin * iw)
            y = int(bboxC.ymin * ih)
            w = int(bboxC.width * iw)
            h = int(bboxC.height * ih)
            cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imshow('Face Detection', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

کتابخانه Face-recognition

کتابخانه‌ی `face_recognition` توسط آقای Ageitgey توسعه داده شده و به عنوان «ساده‌ترین کتابخانه‌ی شناسایی چهره در پایتون» معرفی شده است github.com.

این کتابخانه با استفاده از الگوریتم‌های قدرتمند تشخیص چهره مبتنی بر یادگیری عمیق (سازمان dlib) طراحی شده و امکان تشخیص محل چهره، رمزنگاری چهره و مقایسه چهره‌ها را در تصاویر فراهم می‌کند github.com. به کمک آن می‌توانید تنها با چند خط کد، صورت افراد را شناسایی کرده، ویژگی‌های چهره (مانند نقاط کلیدی چشم و بینی) را استخراج کنید و حتی افراد مختلف را از روی عکس‌ها مطابقت دهید.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install face_recognition
```

توابع و ماژول‌های مهم

توضیح مختصر	تابع / ماژول
خواندن تصویر از فایل و بازگرداندن آرایه‌ی RGB.	<code>face_recognition.load_image_file(filename)</code>
یافتن موقعیت‌های باکس صورت در تصویر (بر حسب پیکسل).	<code>face_recognition.face_locations(image)</code>
محاسبه‌ی بردار رمزنگاری (encoding) ویژگی‌های هر چهره در تصویر.	<code>face_recognition.face_encodings(image)</code>
مقایسه‌ی یک چهره‌ی ناشناس با فهرستی از چهره‌های شناخته‌شده (بر اساس فاصله).	<code>face_recognition.compare_faces(encodings_list, unknown_encoding)</code>
یافتن نقاط کلیدی هر چهره (چشم، بینی، لب و غیره) در تصویر.	<code>face_recognition.face_landmarks(image)</code>

نمونه کد

```
from pyzbar.pyzbar import decode
from PIL import Image

# یا بارکد QR بارگذاری تصویر شامل
img = Image.open("path/to/barcode.png")
decoded_objects = decode(img)

for obj in decoded_objects:
    barcode_data = obj.data.decode("utf-8")
    barcode_type = obj.type
    print(f"نوع کد: {barcode_type}, اطلاعات: {barcode_data}")
```

کتابخانه pyzbar

کتابخانه‌ی **pyzbar** یک رابط پایتون برای کتابخانه‌ی ZBar است که امکان خواندن بارکدها و QR Code ها را در تصاویر فراهم می‌کند pypi.org. این پکیج می‌تواند بارکدهای تک‌بعدی (مانند EAN, CODE128) و بارکدهای دوبعدی (مانند QR Code) را از روی یک تصویر اسکن کرده و اطلاعات رمزنگاری‌شده در آن‌ها را استخراج کند [pyzbar](https://pypi.org). بر روی آرایه‌های NumPy، تصاویر PIL/Pillow و فریم‌های OpenCV کار می‌کند و خروجی را به صورت اشیاء حاوی داده‌های متنی و مختصات مستطیل و رمزگشایی‌شده برمی‌گرداند.

نصب و راه‌اندازی

برای نصب در محیط پایتون از دستور زیر استفاده کنید:

```
pip install pyzbar
```

در برخی سیستم‌ها ممکن است لازم باشد کتابخانه‌ی **zbar** C را هم نصب کنید (مثلاً در اوبونتو: `sudo apt-get install libzbar0`).

توابع و ماژول های مهم

مهم ترین تابع این کتابخانه، `decode` است. استفاده از آن به شکل زیر است:

توضیح مختصر	تابع / ماژول
دریافت یک تصویر (مثلاً از نوع PIL.Image یا آرایه‌ی NumPy) و بازگشت لیستی از اشیاء حاوی نتایج دیکد. هر نتیجه شامل اطلاعاتی مانند <code>data</code> (داده‌ی خوانده شده)، <code>type</code> (نوع کد: QR, EAN, ...) و مختصات مستطیل پیرامون کد است.	<code>pyzbar.decode(image)</code>
رای تعیین نوع نماد قابل استفاده است (مثلاً فقط QR).	<code>pyzbar.pyzbar.ZBarSymbol</code>

نمونه کد

```
from pyzbar.pyzbar import decode
from PIL import Image

# یا بارکد QR بارگذاری تصویر شامل
img = Image.open("path/to/barcode.png")
decoded_objects = decode(img)

for obj in decoded_objects:
    barcode_data = obj.data.decode("utf-8")
    barcode_type = obj.type
    print(f"نوع کد: {barcode_type}, اطلاعات: {barcode_data}")
```


الگوریتم های بینایی

الگوریتم Haar cascade

روش مبتنی بر ویژگی‌های Haar-like با استفاده از طبقه‌بندی‌کننده‌های آبشاری (Cascade Classifier) برای تشخیص اشیاء نظیر چهره و چشم استفاده می‌شود.

- کاربرد: تشخیص سریع چهره/چشم در زمان واقعی
- مزیت: سریع، مناسب برای برنامه‌های آنی
- محدودیت: حساس به تغییرات نور و زاویه تصویر، دقت کمتر نسبت به روش‌های پیشرفته‌تر

الگوریتم SIFT (Scale-Invariant Feature Transform)

الگوریتمی کلاسیک برای استخراج نقاط کلیدی و توصیف‌گرها، مقاوم به تغییر مقیاس، چرخش و نور است.

- کاربرد: پانوراما، تطابق تصویر، ردیابی ویژگی
- مزیت: دقت بالا و پایداری در شرایط تغییر
- محدودیت: محاسباتی سنگین، قدیمی‌تر، و پیش‌از انقضای پتنت محدود بود

الگوریتم SURF (Speeded-Up Robust Features)

نسخه سریع‌تر SIFT است، با استفاده از فیلترهای باکس و تصویر انتگرالی برای افزایش سرعت.

- کاربرد: مشابه SIFT ولی سریع‌تر با کارایی مناسب

- مزیت: سرعت بالا، مناسب برای برنامه‌های بلادرنگ (+)/
- محدودیت: شامل پتنت شده، ممکن است در نسخه رایگان کامل نباشد

الگوریتم LDN (Local Directional Number Pattern)

روشی برای توصیف ویژگی‌های بافتی محلی براساس گرادیانس‌های جهت‌دار (فیلترهای Kirsch).

- کاربرد: تشخیص الگو، تحلیل چهره، طبقه‌بندی بافت
- مزیت: مقاوم نسبت به نویز و تغییر روشنایی
- محدودیت: پیچیدگی پیاده‌سازی بیشتر نسبت به روش‌های ساده

الگوریتم Fisher vector

الگوریتمی برای رمزگذاری توصیف‌گرهای محلی (مانند SIFT) به یک بردار گلوبال قدرتمند برای طبقه‌بندی تصویر.

- کاربرد: طبقه‌بندی تصویر، بازیابی تصویر
- مزیت: کیفیت بالا در یادگیری سنتی و عمیق
- محدودیت: نیاز به مدل آماری پایه مانند GMM

الگوریتم ORB (Oriented FAST and Rotated BRIEF)

الگوریتمی ترکیبی از FAST (برای پیدا کردن نقاط کلیدی) و BRIEF (برای توصیف‌گر باینری)، جایگزینی رایگان و سریع برای SIFT/SURF

- کاربرد: تطابق تصویر، SLAM، ردیابی
- مزیت: سریع، بدون پتنت، مقاوم به نور و چرخش، مناسب بلادرنگ
- محدودیت: دقت کمتر از SIFT/SURF

الگوریتم DCF (Discriminative Correlation Filter)

الگوریتمی برای ردیابی شیء در ویدیو است، با آموزش فیلتر بر اساس نمونه‌های شیء در فریم‌های قبلی.

- کاربرد: ردیابی در زمان واقعی، کاربرد در ویدیوها و دوربین‌های امنیتی
- مزیت: سرعت بالا، پیاده‌سازی منطقی ساده
- محدودیت: در مواجهه با تغییر مقیاس یا چرخش شدید عملکرد پایین‌تر است

خلاصه به صورت جدول

الگوریتم	کاربرد اصلی	مزیت	محدودیت
Haar Cascade	تشخیص چهره	سریع، مناسب Real-Time	حساس به نور و زاویه

SIFT	استخراج نقاط کلیدی	دقت بالا، مقیاس/چرخش بی تاثیر	پتنت دار، سنگین محاسباتی
SURF	مشابه SIFT، سریع تر	سریع تر از SIFT	پتنت دار، پیاده سازی محدود
LDN	تحلیل بافت محلی	مقاوم به نویز و نور	پیچیده تر از LBP
Fisher Vector	تبدیل تصاویر به نمایش برداری	دقت بالا	نیازمند GMM
ORB	کلیدنقاط سریع و تطبیق	سریع، رایگان، مقاوم	کمتر دقیق از SIFT/SURF
DCF	ردیابی اشیاء در ویدیو	سرعت بالا	عملکرد پایین در تغییر شدید

منابع

- وبسایت رسمی scikit-image (معرفی) scikit-image.org
- وبسایت رسمی imageio (مستندات کاربری) imageio.readthedocs.io
- وبسایت رسمی Matplotlib (مستندات و راهنما) github.com
- وبسایت رسمی Albumentations (مستندات و راهنمای نصب) albumentations.ai
- مستندات Torchvision در سایت PyTorch (معرفی دیتاست‌ها و مدل‌ها) docs.pytorch.org
- سایت رسمی quickpose.ai (MediaPipe (Google Developers))
- مخزن GitHub کتابخانه face_recognition (مستندات و مثال‌ها) github.com
- صفحه PyPI/GitHub کتابخانه pyzbar (معرفی و مثال‌ها)
- سایت openCV.org
- سایت geeksforgeeks.org برای برخی از مثال‌های openCV
- سایت wikipedia.org برای تعاریف الگوریتم‌ها
- سایت link.spring.com برای برخی تعاریف الگوریتم‌ها
- سایت berkeley.edu برای برخی تعاریف الگوریتم‌ها